A Machine Learning Framework for Term Deposit Subscription Prediction

Deepali Poudel

DS-630: Machine Learning

Professor Dr. Sharath Kumar Jagannathan

Executive Summary

This project aims to develop a machine learning–based solution to help a Portuguese bank improve the effectiveness of its marketing campaigns for term deposit subscriptions. Historically, the bank relied on repeated phone calls to identify customers likely to subscribe, resulting in high operational costs and low conversion rates. By applying predictive analytics to historic campaign data, we can identify the behavioral, demographic, and economic factors that influence customer decisions. Using the Bank Marketing Dataset—which includes more than 41,000 client interactions recorded between 2008 and 2010—we conducted data cleaning, exploratory analysis, feature engineering, and model development using four supervised learning techniques: Logistic Regression, Random Forests, Gradient Boosting, and XGBoost. Our analysis demonstrates that Gradient Boosting achieves the strongest overall performance, while Logistic Regression and XGBoost offer competitive advantages in recall and balanced classification, respectively. The resulting models allow the bank to target high-value customers more effectively, reduce unnecessary contact attempts, and optimize campaign strategies to increase subscription rates.

Data Description

The dataset, *bank-additional-full.csv*, contains 41,188 records and spans May 2008 to November 2010. Each record represents a unique contact with a client and includes 20 input variables capturing demographic attributes, financial information, campaign details, communication methods, and macroeconomic context.

Demographic and financial attributes include:

- age (numeric), representing the client's age;
- job (categorical), indicating employment type such as admin, technician, services, blue-collar, student, retired, self-employed, or unemployed;
- marital status (married, single, divorced);

- education level (primary, secondary, tertiary, or unknown);
- default, whether the client has credit in default;
- housing and loan, indicating whether the client has a housing or personal loan.

Campaign-specific attributes describe the communication strategy:

- contact, the communication channel (telephone or cellular);
- month and day, reflecting when the last contact occurred;
- duration, the length of the call in seconds;
- campaign, the number of contacts made during the current campaign;
- pdays, the number of days since the client was last contacted as part of a previous campaign (−1 indicates no previous contact);
- previous, how many times the client was contacted in earlier campaigns;
- poutcome, the outcome of previous campaign efforts (success, failure, other, or unknown).

The **target variable** is **y**, indicating whether the client ultimately subscribed to the term deposit (yes/no). Together, these variables provide a comprehensive view of customer behavior, engagement history, and socio-economic context, making the dataset well-suited for predictive modeling.

Methodology

The methodology for this project follows a complete end-to-end machine learning workflow, beginning with data preparation and concluding with model evaluation and interpretation. We first standardized the schema, corrected data types, and encoded categorical variables using one-hot encoding. Missing values were handled through median imputation for numerical fields and mode replacement for categorical fields. Light **feature engineering** was performed by transforming binary variables, consolidating “unknown” categories, and preparing the dataset in a format suitable for

machine learning algorithms. Because the target variable exhibited substantial **class imbalance** with far fewer subscribers than non-subscribers we incorporated imbalance-aware strategies such as class-weighted models to ensure fair and stable learning.

After data cleaning and preprocessing, we conducted Exploratory Data Analysis (EDA) using descriptive statistics and visualizations to uncover behavioral, seasonal, and economic patterns within the data. For predictive modeling, we trained and evaluated four supervised classification algorithms: Logistic Regression, Random Forest, Gradient Boosting, and XGBoost. Model performance was assessed using accuracy, precision, recall, F1-score, and ROC-AUC metrics. Hyperparameters were tuned for each model, and the combined workflow established a rigorous foundation for evaluating customer behavior and supporting data-driven marketing strategy decisions.

Exploratory Data Analysis through Descriptive Statistics and Visualization

INTERPRETATION OF NUMERICAL VARIABLES

Age:

- Mostly concentrated between 25–60, with a smaller elderly segment.
- Older clients tend to show higher interest in long-term deposit products.

Campaign:

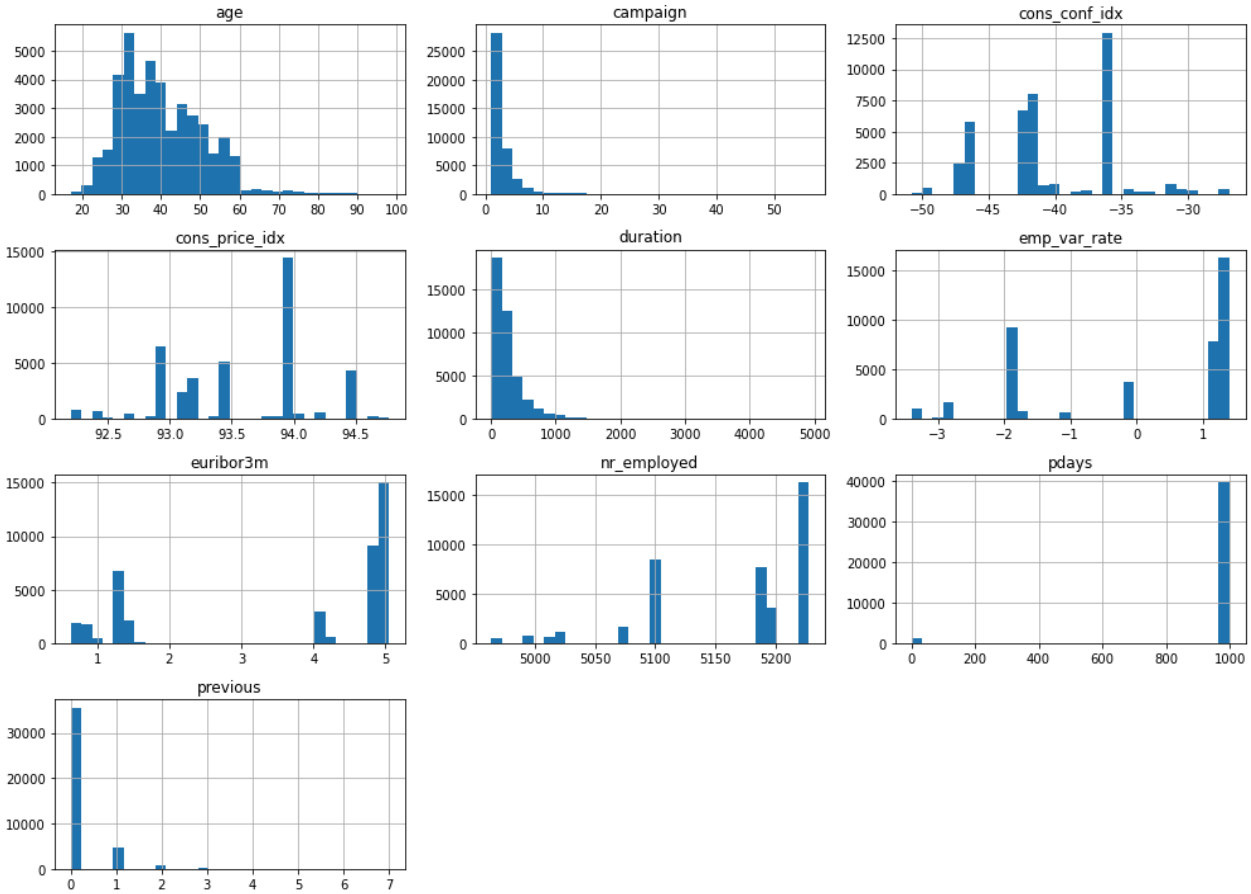
- Strongly right-skewed, with most clients contacted 1- 3 times
- Higher contact counts correlate with lower subscription likelihood due to customer fatigue.

Duration:

- Highly skewed: most calls are short, with a small number lasting several minutes.
- Longer calls strongly correlate with successful subscriptions, reflecting greater engagement.

Pdays:

- Almost all values are 999, meaning clients were not contacted previously.
- Limited direct predictive power, but deviations from 999 are informative.



Previous:

- Most clients have 0 previous contacts, with a small tail up to 7.
- Prior engagement is a meaningful indicator of subscription likelihood.

Economic Indicators (cons_price_idx, cons_conf_idx, euribor3m, nr_employed):

- Clustered distributions reflecting different economic periods between 2008–2010.
- Capture macroeconomic context that may influence customer investment decisions.

Emp_var_rate:

- Exhibits distinct peaks tied to broader economic cycles.

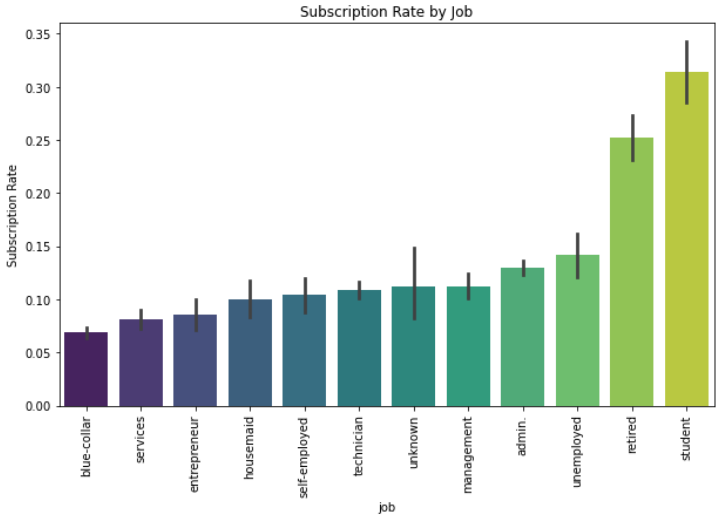
- Strong macro-level indicators often correlated with interest rates and employment levels.

Numeric features show heavy skewness, especially duration, campaign, and pdays. Call-related features capture customer engagement, while economic indicators reflect broader market conditions. Both behavioral and macroeconomic factors are likely to contribute significantly to predictive performance.

INTERPRETATION OF CATEGORICAL VARIABLES

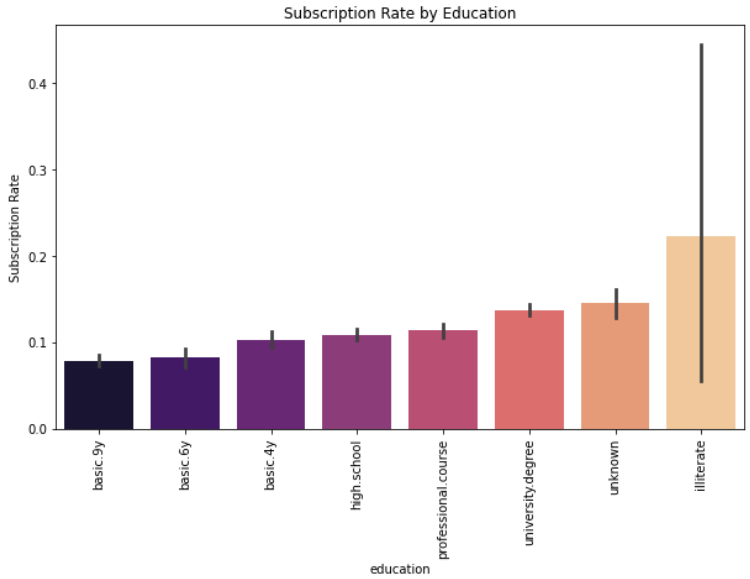
Job

Subscription rates vary noticeably by profession. Students and retirees show the highest likelihood of subscribing, likely reflecting either lower financial obligations (students) or stable long-term financial planning (retirees). Blue-collar and services workers have the lowest subscription rates, suggesting that occupation reflects underlying differences in financial behavior, stability, and interest in long-term deposits.



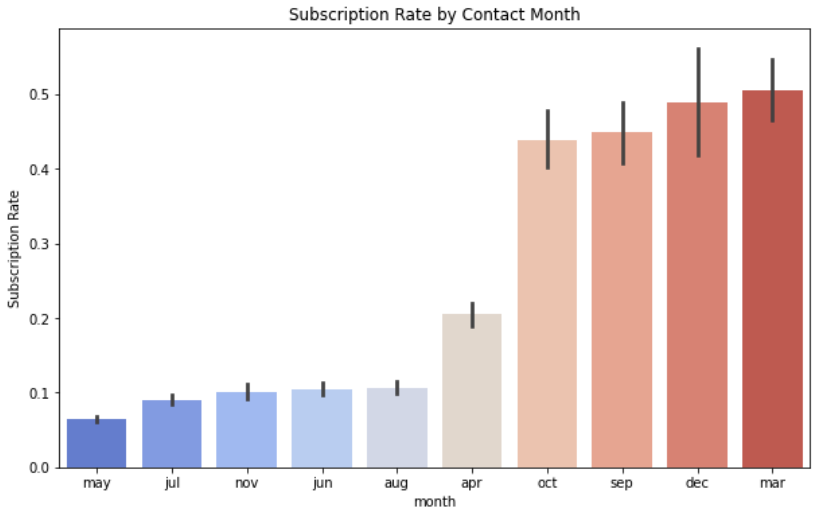
Education

Higher education levels correlate with higher subscription rates. Customers with university or tertiary education show significantly greater interest in term deposits, while primary and unknown education groups have the lowest rates. This pattern likely reflects differences in financial literacy, income stability, and risk preferences.



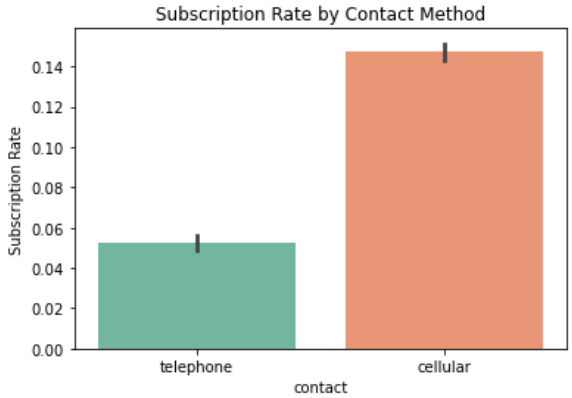
Month of Contact

Subscription rates exhibit clear seasonality. Months such as March, September, October, and December show substantially higher conversion rates, while May, June, and July underperform. This indicates that the timing of outreach plays a strategic role in customer receptiveness, potentially influenced by seasonal financial planning cycles.



Contact Method

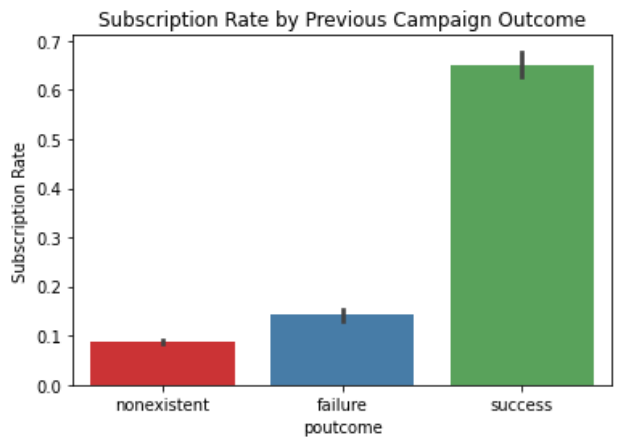
Cellular contact yields markedly higher subscription rates than telephone (landline). This may reflect outdated landline usage or better engagement when customers are contacted via mobile. The choice of communication channel is therefore an operationally critical factor for campaign success.



Previous Campaign Outcome (poutcome)

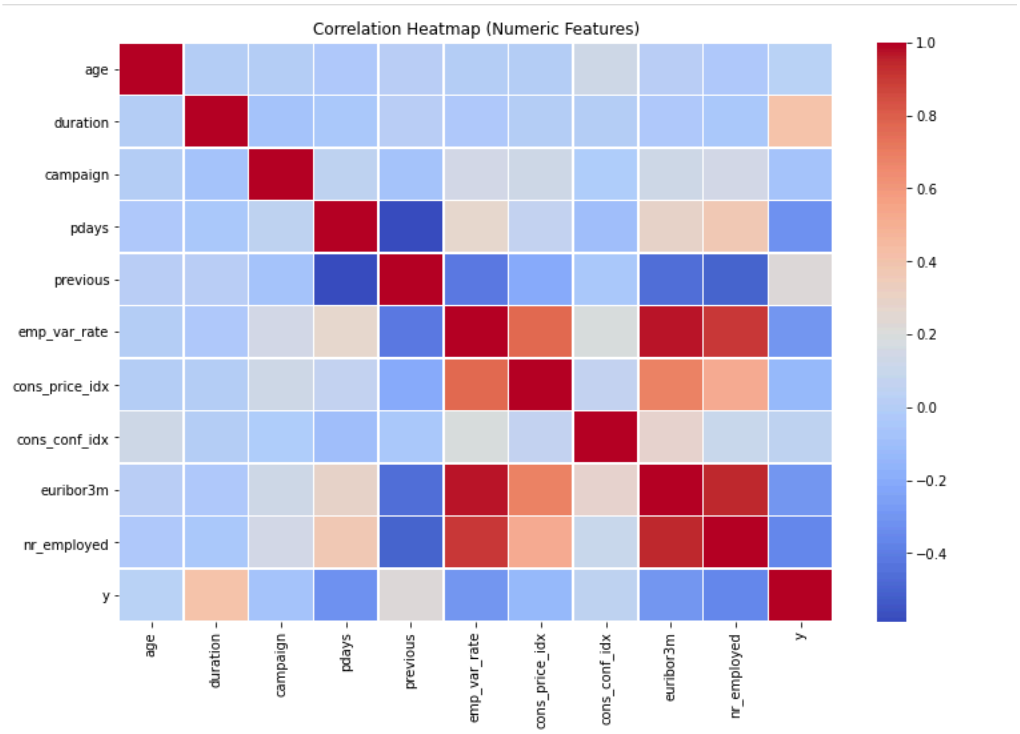
Customers who previously subscribed ("success") demonstrate extremely high conversion rates on subsequent campaigns, confirming them as high-value targets. In contrast, customers with past failures or no prior contact show much lower responsiveness. Prior success acts as a strong indicator of long-term customer loyalty

and financial alignment with the bank.



CORRELATION HEATMAP

The correlation heatmap reveals that numeric features have limited direct linear correlation with the target variable y, and nonlinear models are necessary to capture deeper patterns. However, the exception is notable in duration which shows the strongest positive correlation to y.



Economic indicators such as euribor3m, nr_employed, and cons.price.idx are highly correlated with one another, reflecting underlying macroeconomic trends during the 2008–2010 financial crisis period. These variables exhibit moderate negative correlations with the target, suggesting that economic conditions played a role in customer investment behavior. Features like campaign, pdays, and previous show weak correlations individually, but are known to have strong nonlinear interactions, which explains why tree-based models capture them more effectively.

Modeling and Machine Learning

To evaluate the likelihood of term deposit subscription, four supervised machine learning models were developed and compared: Logistic Regression, Random Forest, Gradient Boosting, and XGBoost. These models were selected to balance interpretability (Logistic Regression), nonlinear pattern detection (Random Forest and Gradient Boosting), and high-performance ensemble learning (XGBoost). All models were trained using an 80/20 train-test split and evaluated with accuracy, precision, recall, F1-score, and ROC-AUC, ensuring robust assessment despite the dataset's significant class imbalance.

Logistic Regression Model (Performance Overview)

Logistic Regression serves as the baseline model due to its transparency and ease of interpretation. It achieved an accuracy of 0.862 and an excellent ROC-AUC of 0.94, indicating strong overall discriminative ability. Because the dataset is imbalanced, recall and precision are more meaningful than accuracy. The model achieved a precision of 0.444, meaning that fewer than half of the predicted subscribers ultimately subscribed—a reasonable outcome given the small number of positive cases. More importantly, it achieved an exceptionally high recall of 0.898, correctly identifying nearly 90% of all true subscribers. This is highly advantageous in a marketing setting, where

the goal is to capture as many potential subscribers as possible. The F1-score of 0.594 reflects this balance, favoring recall.

The confusion matrix further confirms the model's strengths:

```
Confusion Matrix:
[[9402 1563]
 [ 142 1250]]
```

The model produces very few false negatives (142), meaning it rarely misses potential subscribers. Although false positives are higher, this is acceptable in outreach-focused marketing campaigns.

Logistic Regression Feature Interpretation

The model identifies call duration as the strongest predictor of subscription, highlighting the importance of customer engagement. Several macroeconomic variables—such as euribor3m and the consumer price index—also show positive associations with subscription behavior, suggesting that broader economic conditions influence customer receptiveness. Certain months (March, August, September, October) show higher subscription rates, indicating seasonal effects. Prior campaign success increases subscription probability, while repeated contact attempts reduce it due to customer fatigue. Demographically, university-educated, student, and retired clients show greater likelihood of subscribing, while blue-collar and self-employed individuals exhibit lower responsiveness. Mobile contact also performs better than landline communication.

Random Forest (Performance Overview)

The Random Forest model demonstrates strong precision (0.672) but much lower recall (0.425), indicating a conservative prediction pattern that reduces false positives but risks missing many actual subscribers. Its high ROC-AUC of 0.946 shows strong discriminatory capability overall. Random Forest is effective for identifying highly

confident subscriber predictions but is less suitable when capturing most subscribers is the priority.

Gradient Boosting (Performance Overview)

Gradient Boosting emerges as the strongest overall model, achieving the highest accuracy (0.919) and ROC-AUC (0.949) among all four models. Its precision (0.672) and moderate recall (0.542) demonstrate a more balanced profile than Random Forest while retaining excellent model stability. Gradient Boosting is well-suited for general deployment when both reliability and predictive strength are required.

XGBoost (Performance Overview)

XGBoost provides the best balance between precision and recall, producing the highest F1-score (0.632). With a recall of 0.883, comparable to Logistic Regression, and improved precision (0.492), it captures a large proportion of true subscribers while improving prediction quality. Its ROC-AUC of 0.947 further illustrates strong overall performance. XGBoost is ideal when the bank seeks efficient targeting without compromising reach.

Final Model Comparison

	Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Logistic Regression	0.862022	0.444365	0.897989	0.594530	0.939676
1	Random Forest	0.911953	0.672727	0.425287	0.521127	0.946421
2	Gradient Boosting	0.918589	0.672014	0.541667	0.599841	0.948561
3	XGBoost	0.884033	0.491797	0.882902	0.631714	0.947462

All four models deliver strong performance but excel in different areas. Logistic Regression achieves the highest recall, making it ideal for maximizing subscriber

capture in broad outreach strategies. Random Forest provides the highest precision among the simpler models but at the cost of recall. Gradient Boosting offers the strongest overall performance, with the highest accuracy and ROC-AUC, making it the most reliable general-purpose model. XGBoost achieves the best trade-off, delivering the highest F1-score through balanced precision and recall. Together, these results demonstrate that Gradient Boosting is best for overall deployment, Logistic Regression is optimal for maximum capture of potential subscribers, and XGBoost is the most effective hybrid model when both classification quality and coverage matter.

Discussion & Implications

The results of this analysis highlight the importance of aligning marketing strategy with customer engagement patterns, demographic signals, and broader economic trends. The consistently strong predictive power of call duration reinforces the idea that customer engagement quality matters more than outreach volume. Seasonal trends and previous campaign outcomes further suggest that timing and lead history should directly inform how the bank prioritizes its contacts. The models also reveal meaningful demographic differences in subscription behavior, signaling an opportunity to tailor messaging for high-propensity groups such as university-educated, retired, and student customers. Overall, the findings suggest that a targeted, data-driven approach can reduce operational costs, improve conversion rates, and enhance customer experience across future campaigns.

Limitations and Future Work

While the models performed strongly, the analysis is limited by several factors inherent to the dataset. The classes are highly imbalanced, which can affect precision and bias models toward predicting the majority class, even with mitigation strategies such as class-weighting. Additionally, the dataset reflects economic conditions specific to 2008–2010, which may differ significantly from present-day market behavior; thus,

model generalization to modern contexts may be limited. Future work could incorporate more advanced feature engineering, such as interaction terms or temporal features, or test additional algorithms like LightGBM or CatBoost. Moreover, applying techniques such as SMOTE, cost-sensitive learning, or calibration curves could further improve performance, especially in precision-critical contexts.

Conclusion

This project demonstrates that machine learning can effectively support a bank's marketing strategy by predicting which customers are most likely to subscribe to a term deposit. While Logistic Regression excels at identifying the largest number of potential subscribers, Gradient Boosting offers the strongest overall performance, and XGBoost provides the most balanced tradeoff between precision and recall. These complementary strengths enable flexible decision-making depending on campaign goals.

Throughout the project, several technical challenges were encountered, including handling class imbalance, managing categorical encoding, and resolving package compatibility issues—particularly when implementing XGBoost in an older Python environment. Addressing these challenges required adjusting preprocessing steps, applying class-weighted training, and restructuring model inputs to ensure stable performance. These experiences strengthened the overall robustness of the workflow and ensured that the final models were both accurate and reliable.

Ultimately, integrating these models into the bank's outreach process can significantly enhance targeting efficiency, reduce unnecessary contacts, and improve the overall success of future marketing campaigns.

Python Screenshots

[2]:
```python
import pandas as pd

df = pd.read_csv("bank-additional-full.csv", sep=';')
df.head()
df.shape
```

[2]:
```
(41188, 21)
```

[3]:
```python
df.info()
df.describe(include='all')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41188 entries, 0 to 41187
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             41188 non-null  int64
 1   job             41188 non-null  object
 2   marital         41188 non-null  object
 3   education       41188 non-null  object
 4   default         41188 non-null  object
 5   housing         41188 non-null  object
 6   loan            41188 non-null  object
 7   contact         41188 non-null  object
 8   month           41188 non-null  object
 9   day_of_week     41188 non-null  object
 10  duration        41188 non-null  int64
 11  campaign        41188 non-null  int64
 12  pdays           41188 non-null  int64
 13  previous        41188 non-null  int64
 14  poutcome        41188 non-null  object
 15  emp.var.rate    41188 non-null  float64
 16  cons.price.idx  41188 non-null  float64
 17  cons.conf.idx   41188 non-null  float64
 18  euribor3m       41188 non-null  float64
 19  nr.employed     41188 non-null  float64
 20  y               41188 non-null  object
dtypes: float64(5), int64(5), object(11)
memory usage: 6.6+ MB
```

[3]:

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous	poutcome	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed	y
count	41188.00000	41188	41188	41188	41188	41188	41188	41188	41188	41188	...	41188.000000	41188.000000	41188.000000	41188	41188.000000	41188.000000	41188.000000	41188.000000	41188.000000	41188
unique	NaN	12	4	8	3	3	3	2	10	5	...	NaN	NaN	NaN	3	NaN	NaN	NaN	NaN	NaN	2
top	NaN	admin.	married	university.degree	no	yes	no	cellular	may	thu	...	NaN	NaN	NaN	nonexistent	NaN	NaN	NaN	NaN	NaN	no
freq	NaN	10422	24928	12168	32588	21576	33950	26144	13769	8623	...	NaN	NaN	NaN	35563	NaN	NaN	NaN	NaN	NaN	36548
mean	40.02406	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	2.567593	962.475454	0.172963	NaN	0.081886	93.575664	-40.502600	3.621291	5167.035911	NaN
std	10.42125	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	2.770014	186.910907	0.494901	NaN	1.570960	0.578840	4.628198	1.734447	72.251528	NaN
min	17.00000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.000000	0.000000	0.000000	NaN	-3.400000	92.201000	-50.800000	0.634000	4963.600000	NaN
25%	32.00000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.000000	999.000000	0.000000	NaN	-1.800000	93.075000	-42.700000	1.344000	5099.100000	NaN
50%	38.00000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	2.000000	999.000000	0.000000	NaN	1.100000	93.749000	-41.800000	4.857000	5191.000000	NaN
75%	47.00000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	3.000000	999.000000	0.000000	NaN	1.400000	93.994000	-36.400000	4.961000	5228.100000	NaN
max	98.00000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	56.000000	999.000000	7.000000	NaN	1.400000	94.767000	-26.900000	5.045000	5228.100000	NaN

11 rows × 21 columns

[4]:
```python
df['y'].value_counts(normalize=True)
```

[4]:
```
no     0.887346
yes    0.112654
Name: y, dtype: float64
```

[5]:
```python
df.select_dtypes(include='object').columns
```

[5]:
```
Index(['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact',
       'month', 'day_of_week', 'poutcome', 'y'],
      dtype='object')
```

[7]:
```
df.head()
```

[7]:

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous	poutcome	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed	y
0	56	housemaid	married	basic.4y	no	no	no	telephone	may	mon	...	1	999	0	nonexistent	1.1	93.994	-36.4	4.857	5191.0	no
1	57	services	married	high.school	unknown	no	no	telephone	may	mon	...	1	999	0	nonexistent	1.1	93.994	-36.4	4.857	5191.0	no
2	37	services	married	high.school	no	yes	no	telephone	may	mon	...	1	999	0	nonexistent	1.1	93.994	-36.4	4.857	5191.0	no
3	40	admin.	married	basic.6y	no	no	no	telephone	may	mon	...	1	999	0	nonexistent	1.1	93.994	-36.4	4.857	5191.0	no
4	56	services	married	high.school	no	no	yes	telephone	may	mon	...	1	999	0	nonexistent	1.1	93.994	-36.4	4.857	5191.0	no

5 rows × 21 columns

[8]:
```
df['y'] = df['y'].map({'yes': 1, 'no': 0})
```

[9]:
```
df['y'].value_counts(normalize=True)
```

[9]:
```
0    0.887346
1    0.112654
Name: y, dtype: float64
```

[10]:
```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace('.', '_', regex=False)
    .str.replace(' ', '_', regex=False)
    .str.replace('-', '_', regex=False)
)
```

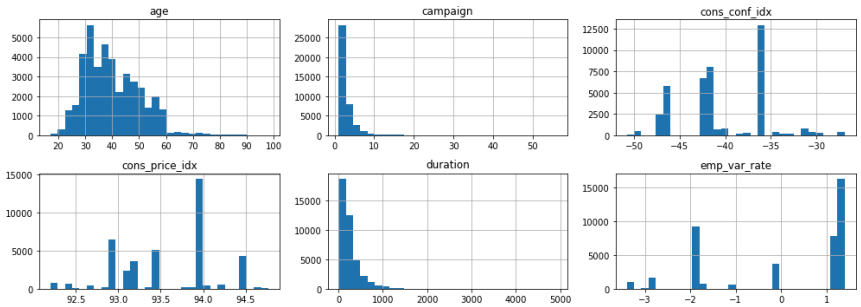
[11]:
```
df.columns
```

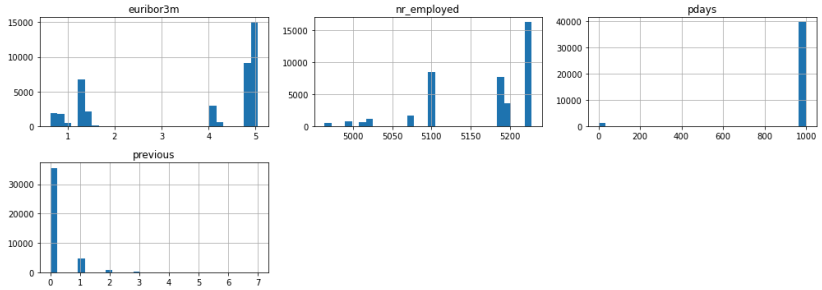
[11]:
```
Index(['age', 'job', 'marital', 'education', 'default', 'housing', 'loan',
       'contact', 'month', 'day_of_week', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'emp_var_rate', 'cons_price_idx',
       'cons_conf_idx', 'euribor3m', 'nr_employed', 'y'],
      dtype='object')
```

[12]:
```
import matplotlib.pyplot as plt

num_cols = [
    'age','duration','campaign','pdays','previous',
    'emp_var_rate','cons_price_idx','cons_conf_idx',
    'euribor3m','nr_employed'
]

df[num_cols].hist(bins=30, figsize=(14,10))
plt.tight_layout()
plt.show()
```





[13]:
```
df.groupby('job')['y'].mean()
df.groupby('education')['y'].mean()
df.groupby('month')['y'].mean()
```

[13]:
```
month
apr    0.204787
aug    0.106021
dec    0.489011
jul    0.090466
jun    0.105115
mar    0.505495
may    0.064347
nov    0.101439
oct    0.438719
sep    0.449123
Name: y, dtype: float64
```

[14]:
```
df.groupby('job')['y'].mean()
```

[14]:
```
job
admin.           0.129726
blue-collar      0.068943
entrepreneur     0.085165
housemaid        0.100000
management       0.112175
retired          0.252326
self-employed    0.104856
services         0.081381
student          0.314286
technician       0.108260
unemployed       0.142012
unknown          0.112121
Name: y, dtype: float64
```

[15]:
```
df.groupby('education')['y'].mean()
```

[15]:
```
education
basic.4y               0.102490
basic.6y               0.082024
basic.9y               0.078246
high.school            0.108355
illiterate             0.222222
professional.course    0.113485
university.degree      0.137245
unknown                0.145003
Name: y, dtype: float64
```

[16]:
```
df.groupby('contact')['y'].mean()
```

[16]:
```
contact
cellular     0.147376
telephone    0.052313
Name: y, dtype: float64
```

[17]:
```
df.groupby('poutcome')['y'].mean().sort_values()
```

[17]:
```
poutcome
nonexistent    0.088322
failure        0.142286
success        0.651129
Name: y, dtype: float64
```

[16]:
```
contact
cellular     0.147376
telephone    0.052313
Name: y, dtype: float64
```

[17]:
```
df.groupby('poutcome')['y'].mean().sort_values()
```

[17]:
```
poutcome
nonexistent    0.088322
failure        0.142286
success        0.651129
Name: y, dtype: float64
```

[18]:
```
df.groupby('marital')['y'].mean().sort_values()
```

[18]:
```
marital
married     0.101573
divorced    0.103209
single      0.140041
unknown     0.150000
Name: y, dtype: float64
```

[19]:
```
df.groupby('housing')['y'].mean().sort_values()
```

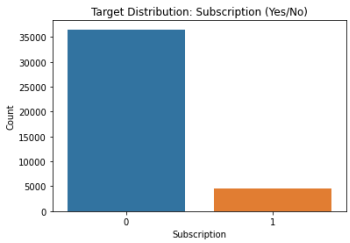
[19]:
```
housing
unknown    0.108081
no         0.108796
yes        0.116194
Name: y, dtype: float64
```

[20]:
```
df.groupby('loan')['y'].mean().sort_values()
```

[20]:
```
loan
unknown    0.108081
yes        0.109315
no         0.113402
Name: y, dtype: float64
```

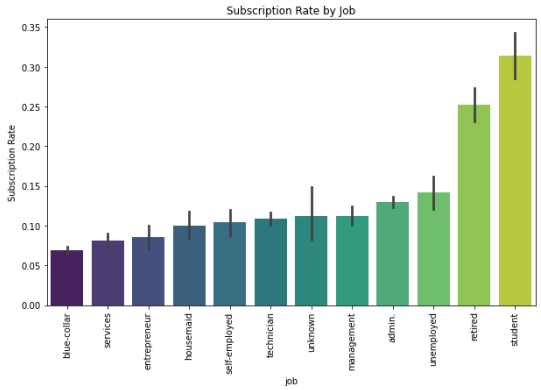
[21]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
sns.countplot(x='y', data=df)
plt.title("Target Distribution: Subscription (Yes/No)")
plt.xlabel("Subscription")
plt.ylabel("Count")
plt.show()
```



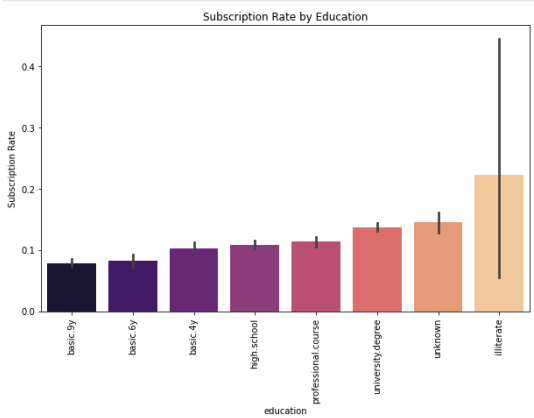
[23]:
```
job_order = df.groupby('job')['y'].mean().sort_values().index

plt.figure(figsize=(10,6))
sns.barplot(x='job', y='y', data=df, order=job_order, palette='viridis')
plt.xticks(rotation=90)
plt.title("Subscription Rate by Job")
plt.ylabel("Subscription Rate")
plt.show()
```



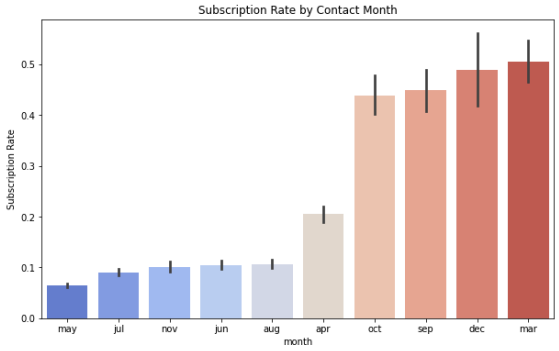
[24]:
```
edu_order = df.groupby('education')['y'].mean().sort_values().index

plt.figure(figsize=(10,6))
sns.barplot(x='education', y='y', data=df, order=edu_order, palette='magma')
plt.xticks(rotation=90)
plt.title("Subscription Rate by Education")
plt.ylabel("Subscription Rate")
plt.show()
```



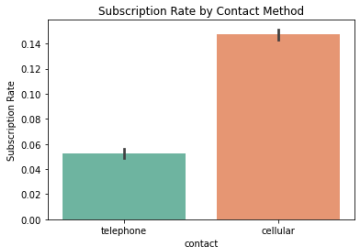
[25]:
```
month_order = df.groupby('month')['y'].mean().sort_values().index

plt.figure(figsize=(10,6))
sns.barplot(x='month', y='y', data=df, order=month_order, palette='coolwarm')
plt.title("Subscription Rate by Contact Month")
plt.ylabel("Subscription Rate")
plt.show()
```



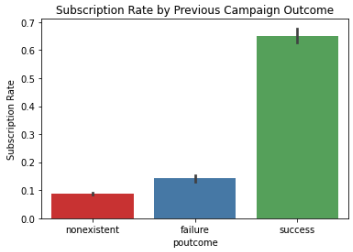
[26]:
```
contact_order = df.groupby('contact')['y'].mean().sort_values().index

plt.figure(figsize=(6,4))
sns.barplot(x='contact', y='y', data=df, order=contact_order, palette='Set2')
plt.title("Subscription Rate by Contact Method")
plt.ylabel("Subscription Rate")
plt.show()
```

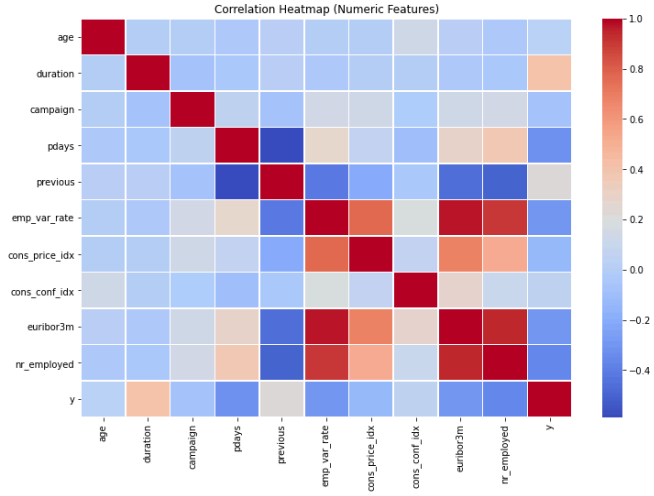


[27]:
```
poutcome_order = df.groupby('poutcome')['y'].mean().sort_values().index

plt.figure(figsize=(6,4))
sns.barplot(x='poutcome', y='y', data=df, order=poutcome_order, palette='Set1')
plt.title("Subscription Rate by Previous Campaign Outcome")
plt.ylabel("Subscription Rate")
plt.show()
```



[28]:
```
plt.figure(figsize=(12,8))
sns.heatmap(df[num_cols + ['y']].corr(), annot=False, cmap='coolwarm', linewidths=.5)
plt.title("Correlation Heatmap (Numeric Features)")
plt.show()
```



[55]:

```python
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_auc_score, classification_report
)

df = pd.read_csv("bank-additional-full.csv", sep=';')

print("Original shape:", df.shape)
print("Original dtypes:\n", df.dtypes, "\n")


obj_cols = df.select_dtypes(include=['object']).columns

for col in obj_cols:
    df[col] = df[col].astype(str).str.strip().str.lower()


numeric_cols = [
    'age', 'duration', 'campaign', 'pdays', 'previous',
    'emp.var.rate', 'cons.price.idx', 'cons.conf.idx',
    'euribor3m', 'nr.employed'
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')


df['y'] = df['y'].map({'yes': 1, 'no': 0})

for col in ['default', 'housing', 'loan']:
    df[col] = df[col].map({'yes': 1, 'no': 0, 'unknown': 0})


num_cols = df.select_dtypes(include=['int64', 'float64']).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

cat_cols = df.select_dtypes(include=['object']).columns
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])

print("Dtypes before dummy encoding:\n", df.dtypes, "\n")


cat_cols = df.select_dtypes(include=['object']).columns
df = pd.get_dummies(df, columns=cat_cols, drop_first=True)

print("Shape after get_dummies:", df.shape)
print("Remaining object columns:", df.select_dtypes(include=['object']).columns.tolist(), "\n")

assert df.select_dtypes(include=['object']).empty, "Still contains non-numeric columns!"


X = df.drop('y', axis=1)
y = df['y']

print("Final X shape:", X.shape)
print("y value counts:\n", y.value_counts())

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

print("Train shape:", X_train.shape, " Test shape:", X_test.shape)
```

```python
X_train = X_train.replace([np.inf, -np.inf], np.nan)
X_test  = X_test.replace([np.inf, -np.inf], np.nan)

train_medians = X_train.median()
X_train = X_train.fillna(train_medians)
X_test  = X_test.fillna(train_medians)


scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)


log_reg = LogisticRegression(
    max_iter=1000,
    class_weight='balanced',
    solver='lbfgs'
)

log_reg.fit(X_train_scaled, y_train)

y_pred  = log_reg.predict(X_test_scaled)
y_proba = log_reg.predict_proba(X_test_scaled)[:, 1]

print("\n=== Logistic Regression Performance ===")
print("Accuracy: ", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall:   ", round(recall_score(y_test, y_pred), 4))
print("F1-score: ", round(f1_score(y_test, y_pred), 4))
print("ROC-AUC:  ", round(roc_auc_score(y_test, y_proba), 4))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred)[:600])
```

```
Original shape: (41188, 21)
Original dtypes:
 age                 int64
job                object
marital            object
education          object
default            object
housing            object
loan               object
contact            object
month              object
day_of_week        object
duration            int64
campaign            int64
pdays               int64
previous            int64
poutcome           object
emp.var.rate      float64
cons.price.idx    float64
cons.conf.idx     float64
euribor3m         float64
nr.employed       float64
y                  object
dtype: object

Dtypes before dummy encoding:
 age                 int64
job                object
marital            object
education          object
default             int64
housing             int64
loan                int64
contact            object
month              object
day_of_week        object
duration            int64
campaign            int64
pdays               int64
previous            int64
poutcome           object
emp.var.rate      float64
cons.price.idx    float64
cons.conf.idx     float64
euribor3m         float64
nr.employed       float64
y                   int64
```

```
Shape after get_dummies: (41188, 51)
Remaining object columns: []

Final X shape: (41188, 50)
y value counts:
 0    36548
1     4640
Name: y, dtype: int64
Train shape: (28831, 50)  Test shape: (12357, 50)

=== Logistic Regression Performance ===
Accuracy:  0.862
Precision: 0.4444
Recall:    0.898
F1-score:  0.5945
ROC-AUC:   0.9397

Confusion Matrix:
[[9402 1563]
 [ 142 1250]]

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.86      0.92     10965
           1       0.44      0.90      0.59      1392

    accuracy                           0.86     12357
   macro avg       0.71      0.88      0.76     12357
weighted avg       0.92      0.86      0.88     12357
```

[56]:

```python
import pandas as pd
import numpy as np

coef_series = pd.Series(log_reg.coef_[0], index=X.columns)

print("\n Top 15 features pushing toward SUBSCRIBING (y=1):")
print(coef_series.sort_values(ascending=False).head(15))

print("\n Top 15 features pushing toward NOT subscribing (y=0):")
print(coef_series.sort_values().head(15))
```

```
 Top 15 features pushing toward SUBSCRIBING (y=1):
duration                        1.821947
euribor3m                       1.460426
cons.price.idx                  1.190876
month_aug                       0.393128
month_mar                       0.224460
poutcome_success                0.167511
poutcome_nonexistent            0.124095
education_university.degree     0.122594
job_retired                     0.105251
marital_single                  0.063492
day_of_week_wed                 0.055366
education_unknown               0.043176
job_student                     0.042330
month_sep                       0.037018
month_oct                       0.029799
dtype: float64

 Top 15 features pushing toward NOT subscribing (y=0):
emp.var.rate         -3.405262
month_may            -0.395215
month_jun            -0.276565
pdays                -0.210980
month_nov            -0.193612
contact_telephone    -0.192116
job_blue-collar      -0.121712
nr.employed          -0.093485
default              -0.087900
campaign             -0.082964
previous             -0.066739
cons.conf.idx        -0.065434
job_self-employed    -0.056747
job_services         -0.051361
education_basic.9y   -0.035968
dtype: float64
```

[57]:
```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix
)

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    class_weight='balanced_subsample',
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train, y_train)   # no scaling for trees

y_pred_rf = rf.predict(X_test)
y_proba_rf = rf.predict_proba(X_test)[:, 1]

print("\n=== RANDOM FOREST Performance ===")
print("Accuracy: ", round(accuracy_score(y_test, y_pred_rf), 4))
print("Precision:", round(precision_score(y_test, y_pred_rf), 4))
print("Recall:   ", round(recall_score(y_test, y_pred_rf), 4))
print("F1-score: ", round(f1_score(y_test, y_pred_rf), 4))
print("ROC-AUC:  ", round(roc_auc_score(y_test, y_proba_rf), 4))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_rf))

# Feature importance
rf_importances = pd.Series(rf.feature_importances_, index=X.columns)
print("\nTop 15 Random Forest features:")
print(rf_importances.sort_values(ascending=False).head(15))
```

```
=== RANDOM FOREST Performance ===
Accuracy:  0.912
Precision: 0.6727
Recall:    0.4253
F1-score:  0.5211
ROC-AUC:   0.9464

Confusion Matrix:
[[10677   288]
 [  800   592]]

Top 15 Random Forest features:
duration              0.392017
euribor3m             0.097497
nr.employed           0.071414
age                   0.059568
emp.var.rate          0.045449
campaign              0.030715
cons.conf.idx         0.028990
cons.price.idx        0.023760
pdays                 0.017966
poutcome_success      0.016784
housing               0.012619
contact_telephone     0.012114
month_may             0.011662
previous              0.011506
marital_married       0.008987
dtype: float64
```

[60]:
```python
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix
)
import pandas as pd


gb = GradientBoostingClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,      # depth of individual trees
    subsample=0.8,
    random_state=42
)

gb.fit(X_train, y_train)

y_pred_gb  = gb.predict(X_test)
y_proba_gb = gb.predict_proba(X_test)[:, 1]

print("\n=== GRADIENT BOOSTING Performance ===")
print("Accuracy: ", round(accuracy_score(y_test, y_pred_gb), 4))
print("Precision:", round(precision_score(y_test, y_pred_gb), 4))
print("Recall:   ", round(recall_score(y_test, y_pred_gb), 4))
print("F1-score: ", round(f1_score(y_test, y_pred_gb), 4))
print("ROC-AUC:  ", round(roc_auc_score(y_test, y_proba_gb), 4))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_gb))

# Feature importance
gb_importances = pd.Series(gb.feature_importances_, index=X.columns)
print("\nTop 15 Gradient Boosting features:")
print(gb_importances.sort_values(ascending=False).head(15))
```

```
=== GRADIENT BOOSTING Performance ===
Accuracy:  0.9186
Precision: 0.672
Recall:    0.5417
F1-score:  0.5998
ROC-AUC:   0.9486

Confusion Matrix:
[[10597   368]
 [  638   754]]

Top 15 Gradient Boosting features:
duration             0.477111
nr.employed          0.237661
euribor3m            0.081124
pdays                0.060012
cons.conf.idx        0.026385
cons.price.idx       0.020809
age                  0.016939
month_oct            0.011908
poutcome_success     0.010325
month_mar            0.010022
contact_telephone    0.007841
emp.var.rate         0.006966
campaign             0.005520
previous             0.004805
day_of_week_mon      0.003391
dtype: float64
```

[61]:
```python
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest", "Gradient Boosting"],
    "Accuracy": [
        accuracy_score(y_test, y_pred),       # from LR
        accuracy_score(y_test, y_pred_rf),    # from RF
        accuracy_score(y_test, y_pred_gb)     # from GB
    ],
    "Precision": [
        precision_score(y_test, y_pred),
        precision_score(y_test, y_pred_rf),
        precision_score(y_test, y_pred_gb)
    ],
    "Recall": [
        recall_score(y_test, y_pred),
        recall_score(y_test, y_pred_rf),
        recall_score(y_test, y_pred_gb)
    ],
    "F1-score": [
        f1_score(y_test, y_pred),
        f1_score(y_test, y_pred_rf),
        f1_score(y_test, y_pred_gb)
    ],
    "ROC-AUC": [
        roc_auc_score(y_test, y_proba),
        roc_auc_score(y_test, y_proba_rf),
        roc_auc_score(y_test, y_proba_gb)
    ]
})

results
```

[61]:

	Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Logistic Regression	0.862022	0.444365	0.897989	0.594530	0.939676
1	Random Forest	0.911953	0.672727	0.425287	0.521127	0.946421
2	Gradient Boosting	0.918589	0.672014	0.541667	0.599841	0.948561

[62]:
```python
from xgboost import XGBClassifier
import numpy as np

X_train_xgb = np.array(X_train, dtype=float)
X_test_xgb  = np.array(X_test, dtype=float)

xgb_model = XGBClassifier(
    n_estimators=400,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    objective='binary:logistic',
    eval_metric='auc',
    scale_pos_weight=(36548/4640),
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train_xgb, y_train)

y_pred_xgb  = xgb_model.predict(X_test_xgb)
y_proba_xgb = xgb_model.predict_proba(X_test_xgb)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred_xgb))
print("Precision:", precision_score(y_test, y_pred_xgb))
print("Recall:", recall_score(y_test, y_pred_xgb))
print("F1:", f1_score(y_test, y_pred_xgb))
print("ROC-AUC:", roc_auc_score(y_test, y_proba_xgb))
```

```
Accuracy: 0.8840333414259124
Precision: 0.491796718687475
Recall: 0.8829022988505747
F1: 0.6317142122847597
ROC-AUC: 0.9474622099574926
```

[63]:
```
new_row = pd.DataFrame({
    "Model": ["XGBoost"],
    "Accuracy": [accuracy_score(y_test, y_pred_xgb)],
    "Precision": [precision_score(y_test, y_pred_xgb)],
    "Recall": [recall_score(y_test, y_pred_xgb)],
    "F1-score": [f1_score(y_test, y_pred_xgb)],
    "ROC-AUC": [roc_auc_score(y_test, y_proba_xgb)]
})

results = pd.concat([results, new_row], ignore_index=True)
results
```

[63]:

	Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Logistic Regression	0.862022	0.444365	0.897989	0.594530	0.939676
1	Random Forest	0.911953	0.672727	0.425287	0.521127	0.946421
2	Gradient Boosting	0.918589	0.672014	0.541667	0.599841	0.948561
3	XGBoost	0.884033	0.491797	0.882902	0.631714	0.947462